AgendaLivre

FATEC POMPEIA

Introdução à Computação em Nuvem · ISW055 · 161_SIST. INTELIGENTES_N

P2

Relatório consolidado

arquitetura e atividades do 2º bimestre

Avaliação individual · 2026.2

Douglas Felipe Mariano

Produto: **AgendaLivre**
https://github.com/Douglas-Felipe-Mariano/TomHanksApp/tree/agendalivre

Prof. Allan Lincoln Rodrigues Siriani
Pompeia, 09/11/2026

Sumário

1 Introdução 2
2 Metodologia 2
3 Arquitetura do SaaS 2
3.1 Visão geral e componentes 2
3.2 Onde roda 3
3.3 Fluxo de uma requisição 4
3.4 Fluxo de assinatura 4
3.5 Modelo de dados do domínio 4
3.6 Segurança 4
3.7 Observabilidade, CI/CD e documentação da API 5
3.8 Camadas IaaS, PaaS e SaaS do projeto 5
4 Quadro de entregas 5
5 Atividades realizadas 6
5.1 Atividade 7 — Plano Premium — cobrança de verdade com Stripe 6
5.2 Atividade 8 — SaaS, PaaS, IaaS — e a documentação que o projeto nunca teve 6
5.3 Atividade 9 — Fork e plano de negócio 7
5.4 Atividade 10 — Seu negócio no ar — o domínio no lugar dos filmes 7
5.5 Atividade 11 — SaaS completo — planos, limites e assinatura com Stripe 8
6 Considerações finais 8
7 Declaração de autoria 9

1 Introdução

A disciplina de Introdução à Computação em Nuvem aborda a construção e operação de aplicações distribuídas, com atenção a identidade, isolamento, dados e infraestrutura. Neste segundo bimestre, o projeto deixou de ser somente um catálogo de filmes e passou a ser o AgendaLivre, um protótipo de SaaS para consultórios independentes. A mudança de domínio reaproveita a base técnica de autenticação, autorização, logs, armazenamento e cobrança, mas substitui as telas e as entidades de negócio.

O AgendaLivre busca reduzir o trabalho manual de organizar horários, pacientes e lista de espera. A persona inicial é a profissional de consultório pequeno que hoje combina agenda, planilha e mensagens para preencher cancelamentos. O sistema guarda apenas dados administrativos mínimos; não é prontuário e não deve conter sintomas, diagnósticos ou recomendações clínicas. Este relatório apresenta a arquitetura que pode ser comprovada no repositório, as atividades 7 a 11, suas datas e estados, e as limitações de publicação e evidências que ainda precisam ser resolvidas.

2 Metodologia

A implementação foi inspecionada na branch `agendalivre` do repositório TomHanksApp. As datas e horas dos commits foram extraídas do Git e conferidas no GitHub; para as atividades 9 e 10, também foi consultado o timestamp de criação das issues. Em 09/11/2026, foi iniciado o frontend Vite local e capturadas a landing page e as telas de login, cadastro, recuperação e redefinição de senha. Como o Docker Engine e MariaDB não estavam disponíveis, esses prints registram apenas a interface e a navegação pública; não comprovam login autenticado, CRUD, deploy, Stripe ou integração de backend.

O enunciado da atividade 9 solicita um repositório novo, público e espelhado com todo o histórico, além do backlog. Por decisão de continuidade, o AgendaLivre permaneceu no mesmo repositório, na branch `agendalivre`; essa divergência está explicitada na issue 7 e não é apresentada como requisito cumprido. Não tive acesso verificável ao Portainer, à conta Stripe de teste nem ao banco MariaDB remoto nesta execução. Portanto, capturas de checkout, webhook, limite Free em integração, stack saudável e URL implantada não foram inventadas nem substituídas por imagens ilustrativas.

3 Arquitetura do SaaS

3.1 Visão geral e componentes

O diagrama a seguir representa os componentes e as integrações declaradas no código e no Docker Compose. Ele não comprova que a stack esteja implantada ou saudável na VM da disciplina.

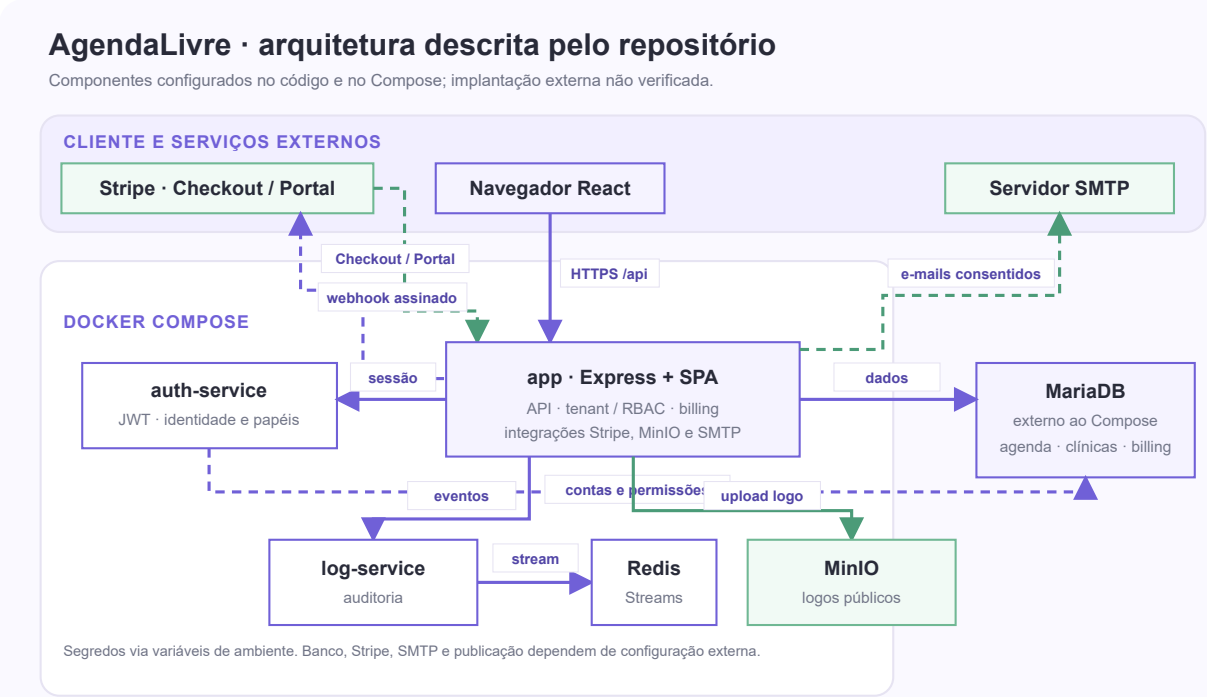


Figura 1: AgendaLivre — arquitetura configurada no repositório; implantação externa ainda não verificada.

Componente	Responsabilidade	Tecnologia / configuração	Exposição
app	Landing, SPA, API do domínio e integrações	Node.js / Express; porta 3000	público via domínio
auth-service	Cadastro, login, validação de token e papel global	Node.js; porta interna 3001	rede Docker
log-service + Redis	Eventos de auditoria e leitura administrativa	Node.js; Redis Streams	rede Docker
MariaDB	Contas, clínicas, equipe, pacientes, agenda e billing	Externo ao Compose; `DB_*`	acesso da aplicação
MinIO	Armazenamento de logos do consultório	bucket `agendalivre-assets`	API interna; objeto público
Stripe	Checkout, assinatura, portal e webhooks	API Stripe em Test mode	serviço externo
SMTP	E-mail de recuperação e campanhas consentidas	`SMTP_*`	serviço externo

3.2 Onde roda

O repositório contém Dockerfiles e um `docker-compose.yml` para os containers `app`, `auth-service`, `log-service`, Redis e MinIO. O MariaDB não é criado pelo Compose: endereço, usuário, senha e banco são fornecidos por `DB_HOST`, `DB_USER`, `DB_PASS` e `DB_NAME`. O app serve o build do frontend e expõe a porta configurada por `RESERVED_PORT`; Stripe e SMTP são dependências externas. O bucket do MinIO destina-se somente a logos públicos.

Não há prova disponível nesta execução de qual stack do Portainer está ativa, do repositório configurado nela, dos limites de memória efetivos, do domínio público ou do estado dos containers. Assim, não declaro uma URL de produção. A execução que foi observada para conferir a interface foi local; não equivale ao deploy solicitado nas atividades 9 e 10.

3.3 Fluxo de uma requisição

1. O navegador envia uma requisição autenticada ao endpoint `/api` do `app`.
2. O middleware de autenticação valida a identidade consultando `/auth/me` no `auth-service`; o papel global atual é lido do banco.
3. `clinicAccess` resolve no servidor o consultório e o papel local (`owner`, `reception` ou `professional`). O tenant não é escolhido por um ID fornecido pelo navegador.
4. A rota aplica o papel permitido e restringe consultas pelo `clinic_id`; algumas leituras de profissionais também usam o nome da sessão, limitação documentada que deve evoluir para vínculo por ID imutável.
5. O `app` consulta ou altera o MariaDB e registra eventos no `log-service`, que os guarda no Redis Streams. A resposta JSON retorna pelo mesmo caminho ao navegador.

3.4 Fluxo de assinatura

O backend cria uma Checkout Session hospedada pelo Stripe com `mode=subscription`, vinculada à conta por `client_reference_id` e metadados. O cartão nunca passa pelo backend. A rota exige o preço `STRIPE_PRO_PRICE_ID` ativo em BRL, mensal e de R$ 39,90. Na volta do Checkout, o frontend apenas consulta o estado; somente os webhooks validados podem sincronizar a assinatura e conceder Pro.

O webhook usa o corpo bruto e `Stripe-Signature`. Os IDs de evento são gravados em `stripe_events` na mesma transação da alteração do usuário, evitando reaplicar um evento repetido. Estados `active`/`trialing` concedem Pro; cancelamento ou `past_due` retiram o benefício. O endpoint do Customer Portal delega gerenciamento/cancelamento ao Stripe. A configuração efetiva do Dashboard, a entrega dos webhooks e o ciclo Checkout⟶banco⟶limite não foram testados com credenciais reais de teste. Eventos fora de ordem do mesmo ID de assinatura ainda podem sobrescrever o estado e são limitação conhecida.

3.5 Modelo de dados do domínio

As tabelas centrais são `usuarios` (papel global, estado Pro e IDs Stripe), `clinics` (tenant e proprietário), `clinic_members` (vínculo único de equipe por usuário e papel local), `clinic_patients`, `clinic_appointments`, `clinic_waitlist`, `clinic_marketing_sends`, `reset_tokens` e `stripe_events`. As relações e chaves estrangeiras são declaradas em `database/create.sql` e repetidas/adequadas pelas migrações do backend. `clinic_id` liga os dados operacionais ao consultório; `patient_id` liga atendimento/lista de espera ao paciente; `owner_user_id` liga a clínica à conta proprietária.

Os campos `stripe_customer_id` e `stripe_subscription_id` relacionam a conta ao Stripe; `premium`, `subscription_status`, `subscription_amount_cents`, `premium_since` e `premium_until` guardam a cópia operacional local do plano. Essa cópia depende de webhooks e não é a fonte autoritativa da cobrança.

3.6 Segurança

O papel `admin` é global e permite acessar o resumo administrativo de billing e os logs. `owner` administra clínica, equipe, configuração e cobrança; `reception` consulta a operação e pode cadastrar pacientes, consentimentos e lista de espera; `professional` cria e administra atendimentos associados ao próprio nome de sessão. O backend aplica as regras; esconder controles na interface não é considerado autorização.

As consultas do domínio usam `clinic_id`, e alterações de paciente/consulta verificam que o registro pertence ao tenant. Para profissionais, várias leituras e mutações também filtram pelo nome, não por uma chave imutável, e por isso a documentação registra esse risco. Segredos Stripe, JWT, SMTP, banco e HMAC são nomes

de variáveis nos arquivos de exemplo; valores reais não devem ser versionados. O bucket MinIO é público para logos e não deve conter dados de pacientes. Rate limiting, revisão LGPD, retenção e hardening de produção ainda são necessários.

3.7 Observabilidade, CI/CD e documentação da API

O `log-service` usa Redis Streams para eventos e a API administrativa limita o acesso ao papel `admin`. Há testes unitários locais para helpers de billing. Não foi implementado nem validado nesta entrega pipeline de CI/CD, Swagger/OpenAPI, métricas Prometheus ou health checks/readiness de ponta a ponta. A stack não foi observada no Portainer.

3.8 Camadas IaaS, PaaS e SaaS do projeto

Camada	Papel neste projeto	Gestão / estado verificado
IaaS	VM, sistema operacional, disco e rede do laboratório	Infraestrutura da Fatec/provedor; implantação individual não verificada.
PaaS (didático)	Docker/Compose e containers da aplicação, auth, logs, Redis e MinIO	Configurados no repositório; stack/limites no Portainer não inspecionados.
SaaS	AgendaLivre para o consultório; Stripe e SMTP como serviços externos	Produto é protótipo; integração externa não validada nesta execução.

4 Quadro de entregas

Nº	Atividade	Planejada	Realizada	Situação
7	Plano Premium — cobrança de verdade com Stripe	09/10/2026	09/11/2026 21:01	implementação parcial, com atraso; evidência de integração ausente
8	SaaS, PaaS, IaaS — e a documentação que o projeto nunca teve	16/10/2026	09/11/2026 22:29	documentação commitada com atraso; publicação da stack não verificada
9	Fork e plano de negócio	23/10/2026	09/11/2026 22:30	não entregue conforme enunciado; issue de

Nº	Atividade	Planejada	Realizada	Situação
				divergência aberta
10	Seu negócio no ar — o domínio no lugar dos filmes	06/11/2026	09/11/2026 22:30	código parcial commitado com atraso; deploy e evidências pendentes
11	SaaS completo — planos, limites e assinatura com Stripe	13/11/2026	Em andamento — código commitado em 09/11/2026 22:29	em andamento; ainda não demonstrada em Stripe/ MariaDB de teste

5 Atividades realizadas

5.1 Atividade 7 — Plano Premium — cobrança de verdade com Stripe

Descrição	Checkout de pagamento hospedado e sincronização do benefício Premium via webhook.
Data planejada	09/10/2026
Data realizada	09/11/2026 21:01
Situação	implementação parcial, com atraso; evidência de integração ausente
Evidência disponível / pendente	Commit 70586a6 e testes automatizados locais. Capturas do checkout concluído, webhook e benefício não disponíveis.
Link direto	https://github.com/Douglas-Felipe-Mariano/TomHanksApp/commit/70586a659c9f78db3f5e30f5063f0f41e337f16a

O commit `70586a6` contém a primeira integração de Checkout Premium com o Stripe e processamento de webhook. O teste local cobre resolução da sessão e regras de elegibilidade; não comprova uma compra de teste executada no Dashboard, uma alteração efetiva no MariaDB ou o benefício em um ambiente publicado. Essa parte foi feita depois da data planejada de 09/10.

A pendência é reunir as capturas exigidas pelo enunciado: Checkout de teste concluído e a conta após o webhook, além do benefício funcionando. Elas não foram produzidas nesta execução porque não há acesso configurado ao Stripe e ao MariaDB de teste. A decisão de delegar cartão ao Checkout evita armazenar dados de cartão no produto.

5.2 Atividade 8 — SaaS, PaaS, IaaS — e a documentação que o projeto nunca teve

Descrição	Classificação das camadas e documentação reversa da especificação, arquitetura, regras e dados.
Data planejada	16/10/2026

Data realizada	09/11/2026 22:29
Situação	documentação commitada com atraso; publicação da stack não verificada
Evidência disponível / pendente	Commit 0f754a8; README e DOCUMENTACAO.md no branch. Sem captura de implantação.
Link direto	https://github.com/Douglas-Felipe-Mariano/TomHanksApp/commit/0f754a8c88289d29030b93d245a35073f161cb99

A documentação foi atualizada para o AgendaLivre: o README apresenta a dor, persona, alternativas, receita, métricas, Lean Canvas e mapa de reaproveitamento. `DOCUMENTACAO.md` registra API, arquitetura, regras, modelo de dados, permissões e limitações. O documento separa fatos observados no código das dependências externas ainda não verificadas.

O commit é posterior ao prazo de 16/10/2026. Não há captura de um deploy para anexar; a atividade 8 de documentação pode ser consultada diretamente no commit e nos arquivos. A principal decisão foi declarar limitações em vez de descrever uma infraestrutura ideal como se estivesse em execução.

5.3 Atividade 9 — Fork e plano de negócio

Descrição	Repositório novo espelhado, README do produto e backlog de pelo menos cinco issues.
Data planejada	23/10/2026
Data realizada	09/11/2026 22:30
Situação	não entregue conforme enunciado; issue de divergência aberta
Evidência disponível / pendente	Issue #7 registra a pendência. O README existe neste repositório; não há repositório separado nem evidência de espelhamento.
Link direto	https://github.com/Douglas-Felipe-Mariano/TomHanksApp/issues/7

O README do AgendaLivre descreve um produto para consultórios, dor e persona, alternativas, proposta de valor, Free/Pro, métricas, Lean Canvas e reaproveitamento dos microsserviços. A implementação, porém, permaneceu no repositório existente, na branch `agendalivre`, em vez de ser espelhada para um repositório novo e público. Isso contraria um requisito explícito da atividade 9 e não é apresentado como concluído.

A issue 7 foi aberta em 09/11/2026 às 22:30 (horário de Brasília) para registrar a divergência e pedir uma decisão sobre como regularizar o requisito. O README e o histórico existem, mas não há captura do histórico em repositório novo. O backlog de produto também precisa ser distinguido das issues de entrega acadêmica e completado com pelo menos cinco próximas tarefas.

5.4 Atividade 10 — Seu negócio no ar — o domínio no lugar dos filmes

Descrição	CRUD do domínio, papéis, isolamento, eventos, upload e landing page na stack do curso.
Data planejada	06/11/2026
Data realizada	09/11/2026 22:30
Situação	código parcial commitado com atraso; deploy e evidências pendentes
Evidência disponível / pendente	Issues #8 e commits 45ef84a/5b03adc. A URL pública, o 403 integrado, MinIO e logs não foram validados.

Link direto	https://github.com/Douglas-Felipe-Mariano/TomHanksApp/issues/8

A branch substitui o catálogo por uma landing page pública, painel de consultório, pacientes, agenda, lista de espera, papéis locais, isolamento por `clinic_id`, eventos e upload de logo. Após harmonizar a identidade visual do login com a landing, o frontend foi iniciado e as telas públicas foram capturadas às 23:03 de 09/11/2026 em Brasília: landing, login, cadastro, recuperação e redefinição sem token. São telas observadas no navegador local, não demonstração de operações persistidas. O Docker Engine não estava disponível, não há `.env` nem MariaDB local e o backend não foi iniciado, então não foi possível autenticar, criar registros ou capturar os fluxos protegidos. A issue 8 foi aberta em 09/11/2026 às 22:30, depois do prazo de 06/11, e enumera as validações ainda necessárias.

Não tive acesso ao Portainer nem à base MariaDB remota. Portanto, não posso declarar a stack redeployada, os containers saudáveis, o subdomínio atualizado ou os testes de duas contas concluídos. Faltam os prints exigidos da landing no domínio, do 403, dos eventos, do MinIO e do Portainer; o endereço local do navegador não é evidência de publicação.

5.5 Atividade 11 — SaaS completo — planos, limites e assinatura com Stripe

Descrição	Planos Free/Pro, assinatura recorrente, webhooks, Customer Portal, limites server--side e MRR admin.
Data planejada	13/11/2026
Data realizada	Em andamento — código commitado em 09/11/2026 22:29
Situação	em andamento; ainda não demonstrada em Stripe/MariaDB de teste
Evidência disponível / pendente	Commit 4660962 e 5 testes locais. Capturas de Checkout, webhook, bloqueio Free e painel admin pendentes.
Link direto	https://github.com/Douglas-Felipe-Mariano/TomHanksApp/commit/4660962e16826ce41161a848b24c024f4fd152b3

O código cria Checkout de assinatura com preço mensal fixado no servidor, guarda IDs da assinatura/cliente, valida assinatura dos webhooks, deduplica eventos, sincroniza o plano, expõe o Customer Portal, aplica no backend a franquia Free de três agendamentos e calcula o MRR no painel admin. Os cinco testes unitários de billing passaram localmente. A data planejada de 13/11 ainda não chegou na data desta versão do relatório.

Isso não comprova o cenário ponta a ponta exigido: não há preço, Customer Portal, segredo de webhook, conta de teste ou banco integrado configurados nesta execução. Assim, assinatura com cartão 4242, evento no Stripe, limite bloqueando em conta Free, cancelamento e MRR real continuam pendentes. Não afirmo a atividade como entregue.

6 Considerações finais

O trabalho deste bimestre mostrou que mudar o domínio de uma aplicação envolve mais do que trocar telas: foi necessário modelar tenants, definir papéis locais, controlar os acessos no servidor e manter o estado de assinatura separado da resposta do navegador. A parte que mais exige cuidado é provar que as integrações e a infraestrutura configurada no código realmente funcionam em conjunto, sem confundir build local ou testes unitários com um deploy real.

Neste momento, o AgendaLivre tem uma base de produto e código funcional localmente, mas este relatório não deve ser tratado como evidência de um SaaS publicado. Para concluir a entrega faltam resolver o requisito de repositório da atividade 9, validar e publicar as atividades 10 e 11 nos serviços da disciplina, anexar capturas

originais e atualizar as issues com os resultados. Uma futura versão deve também vincular profissionais por ID, padronizar timezone e tornar a cota de e-mails atômica.

7 Declaração de autoria

Declaro que este relatório foi elaborado por mim, individualmente. As datas de commits, issues e testes citadas são verificáveis nos links informados. As capturas de ambiente externo ainda não disponíveis foram declaradas como pendentes, sem simulação de evidência.

Douglas Felipe Mariano	Pompeia, 09/11/2026